

— TECHNICAL PRIMER

yasqat

sequence analytics, native to Python

A polars-native toolkit for categorical sequence analysis — its data model, architecture, capabilities, and roadmap, for technical leadership.

rexarski

v0.5.0 live on PyPI · 2026.07

github.com/rexarski/yasqat · rexarski.github.io/yasqat · MIT License

| Contents

01	Overview & Lineage	03
02	The Problem & the Approach	04
03	Data Model & Architecture	05
04	Capabilities	07
05	Design Principles & Engineering	08
06	Status & Roadmap	09
07	Appendix — Glossary & References	10

01 · OVERVIEW

Overview & Lineage

yasqat (Yet Another Sequence Analytics Toolkit) is a Python library for **categorical sequence analysis** — the study of ordered categorical states observed over time. It is built on polars and aimed at life-course research: labour-market, health, and education trajectories.

What "categorical sequence analysis" means

Each unit of study — a person, an account, a patient — is represented as a sequence: an ordered run of states drawn from a small, fixed vocabulary (an *alphabet*), for example `{employed, unemployed, retraining}`. The discipline asks three recurring questions of a population of such sequences:

- Similarity — how alike are two trajectories? This needs a principled distance.
- Typology — what distinct "kinds" of trajectory exist? This needs clustering.
- Dynamics — how do states flow, how diverse or volatile is a path over time?

Lineage

The reference implementation for the field is TraMineR (R) — yasqat treats it as the oracle for algorithm correctness, naming, and feature scope. TanaT (Python) is a secondary reference for Python-specific adaptations. The goal is not novelty in the algorithms but fidelity to proven methods, delivered in a modern Python form.

QUESTIONS THIS DOCUMENT ANSWERS

What is yasqat and where do its methods come from? Why build it on polars rather than reuse R? How is it structured, what can it do today, and what changes for integrators as it moves toward 1.0?

02 · PROBLEM & APPROACH

| The Problem & the Approach

The methods are settled; their *availability* is not. The canonical toolkit lives in R, while a growing share of applied data work happens in Python on columnar, Arrow-era stacks.

Current state

TraMineR is mature and trusted, but it is R-bound. Teams working in Python must either bridge across the language boundary or reimplement piecemeal. Neither path composes cleanly with the columnar, multithreaded data tooling — Arrow, polars, Spark — that now underpins production pipelines. The result is friction exactly where sequence analysis would otherwise plug into feature engineering and modelling.

The core problem

There has been no *native, fast, DataFrame-first* sequence-analysis library for Python that a practitioner can drop into an existing pipeline without an R round-trip or a custom port.

The approach

yasqat takes a deliberately narrow set of technical positions:

- polars-native — columnar, multithreaded, lazy-capable. polars is the only DataFrame in core code; pandas never appears there (pyarrow serves only as an interop bridge).
- numba-JIT inner loops — the hot pairwise kernels in `METRICS` are compiled with `@njit` over integer-encoded arrays; scipy supplies hierarchical linkage.
- DataFrame in, DataFrame out — every public method returns a polars `DataFrame`. Users bring their own plotting library.

A standing design contract: every public method returns a polars `DataFrame`; users bring their own plotting tool. No visualization layer is bundled, so yasqat never competes with — or locks you out of — matplotlib, altair, or anything else.

03 · DATA MODEL & ARCHITECTURE

Data Model & Architecture

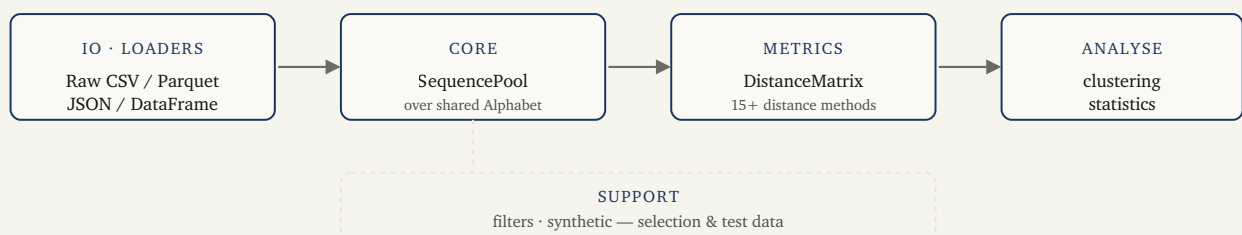
Four value types carry the domain, and a one-directional pipeline moves data through seven modules. Each module owns one concept; data only ever flows forward.

The core types

Type	Role
Alphabet	The finite state vocabulary — encoding to/from integers, plus a colour map for consistent downstream palettes.
SequenceConfig	Column configuration (which columns are id / time / state) — keeps the data model declarative.
StateSequence	A typed container over a long-format frame: format conversions (STS / SPS / DSS) and descriptive aggregations (durations, spells, spans).
SequencePool	The analysis engine: pre-extracted per-id state lists for fast random access, and the entry points for distances, clustering, sampling, and recoding.

The pipeline

Raw data enters through loaders, becomes a pool over a shared alphabet, is scored pairwise into a distance matrix, and then feeds clustering and statistics. Support modules sit to the side.



Data flows one direction: load → represent → measure → analyse. Support modules feed selection and testing.

The module map

Module	Responsibility
CORE	Alphabet, StateSequence, SequencePool, SequenceConfig — the data model.
METRICS	Pairwise distance metrics → DistanceMatrix; the substitution-cost builder.
STATISTICS	Transition rates, descriptive & normative indicators, subsequence mining, discrepancy trees.
CLUSTERING	PAM, CLARA, hierarchical; quality indices and representatives.
FILTERS	Sequence filtering criteria (length, time, state, query).
IO	Loaders & savers: CSV / Parquet / JSON / DataFrame ↔ sequence types.
SYNTHETIC	Markov and financial trajectory generators for tests and demos.

A minimal end-to-end example

```

from yasqat.io import load_csv
from yasqat.clustering import pam_clustering
from yasqat.statistics import longitudinal_entropy

# Every loader returns a SequencePool — the canonical analysis container
# (ADR-0002). Default columns are id/time/state.
pool = load_csv("trajectories.csv")

dm      = pool.compute_distances(method="om", indel=1.0, n_jobs=4) #
DistanceMatrix
result = pam_clustering(dm, n_clusters=4)      # a trajectory typology

longitudinal_entropy(pool)                    # → a polars DataFrame, like every
result

```

04 · CAPABILITIES

| Capabilities

Roughly 90 public functions and classes across the seven modules. The breadth tracks TraMineR's feature scope; the table below is the working inventory.

Area	What's available
Distance metrics	Optimal Matching and its variants (OMloc, OMspell, OMstran, NMS, NMSMST, SVRspell), Hamming, LCS, LCP / RLCP, DTW, SoftDTW, Chi ² , Euclidean, DHD, TWED — with length/similarity convenience wrappers.
Substitution costs	Constant, transition-rate-derived, INDEL and INDEL-log, "future" (chi-squared), and feature-based (Gower distance).
Clustering	PAM (k-medoids, with <code>.predict()</code>), CLARA, hierarchical (scipy linkage); parallel pairwise distance computation via <code>n_jobs</code> .
Cluster quality	Average silhouette width (ASW), Point Biserial, Hubert's Gamma, R ² , PAM range analysis, distance-to-center, representative extraction.
Descriptive statistics	Longitudinal entropy, transition rates, complexity index, turbulence, spell counts, visited & modal states (with time granularity), sequence frequencies, log-probabilities, subsequence counts.
Normative indicators	Volatility (label-weighted and label-free objective variants), precarity, insecurity, degradation, badness, integration (per-state), proportion-positive, per-sequence state distributions — for "good vs. bad state" social-science measures.
Discrepancy & mining	Pseudo-ANOVA with permutation tests, multi-factor discrepancy, dissimilarity trees; frequent-subsequence mining with support and length thresholds; sequential association rules scored by confidence, lift, leverage, and conviction.
I/O & synthetic	CSV / Parquet / JSON / DataFrame load & save with type inference (Arrow / Hive / Spark interop); Markov and financial sequence generators.

05 · DESIGN PRINCIPLES

| Design Principles & Engineering

The codebase favours **deep modules behind narrow seams**: a lot of capability reached through a small, obvious interface. Four seams carry most of the surface.

Single seams, not scattered entry points

- One metric seam. A metric is a free function `name_distance(seq_a, seq_b, **kwargs) → float` over integer-encoded arrays. Every metric is registered in one dispatch dictionary inside `SequencePool.compute_distances(method=...)` — the sole path for pairwise and matrix computation.
- One encoding seam. States become integers in exactly one place, `get_encoded_sequence`, so the JIT kernels always see a consistent representation.
- One coercion seam. Statistics and filters accept either container through a `SequenceData` protocol and normalize via `SequencePool.coerce` / `StateSequence.coerce` — both delegating to a single rebuild rule — no scattered type checks.
- One per-sequence reduce seam. Every indicator that maps a scalar over each sequence goes through `reduce_per_sequence`, which owns the loop, the id column, and the per-sequence / aggregate collapse; each statistic contributes only its scalar function.

Hardening shipped in v0.5.0

The v0.5.0 cycle was a deliberate simplification pass — removing concepts rather than adding them, so that each task has exactly one obvious path:

- Removed a dead per-metric class layer (an unused abstract base and eleven subclasses), collapsing all distance work onto `compute_distances`.
- Removed a dead encoding method that no caller used.
- Unified the statistics layer on the `SequenceData` protocol and the symmetric `coerce` seam, deleting ~30 duplicated type-coercion blocks.
- Collapsed seventeen loop-shaped per-sequence statistics onto the `reduce_per_sequence` seam — each is now just a scalar function, with the boilerplate written once.

The throughline of v0.5.0 is **subtraction**: fewer types, fewer branches, one seam per job. Less surface to learn, and less surface to break.

Engineering discipline

The project enforces `ruff` (lint + format, 88-column, py311 target) and `mypy` in strict mode on every change; tests pin exact expected values rather than asserting loose bounds. Releases publish to

PyPI from CI on a version tag via OIDC. Architectural decisions are recorded as ADRs, and the issue tracker lives in-repo as markdown — so the project's reasoning travels with the code.

| Status & Roadmap

yasqat is pre-1.0 and follows semantic versioning accordingly: while breaking changes are still possible, they land in *minor* bumps and are documented. Here is the current arc and what it means if you depend on the library.

Version	Status	What it means for integrators
v0.4.1	Superseded	The previous release; upgrading is a <code>pip install -U</code> away (see migration note below).
v0.5.0	Live on PyPI	The current installable release; the full feature set described here, including association rules, per-sequence state distributions, and objective volatility.
1.0	Planned	A stable, committed public interface once the data-model vocabulary is settled.

Migration note: upgrading to v0.5.0

The breaking changes affect direct users of internal classes, not typical pipeline code:

- The per-metric classes are gone. Replace `SomeMetric(...).compute(a, b)` with the free function `some_distance(a, b, ...)`, and matrix building with `pool.compute_distances(method=...)`.
- `StateSequence.encode_states()` is removed; encode via `alphabet.encode(...)` or per-sequence through the pool.

Open questions on the path to 1.0

The container question is now settled: `SequencePool` is the canonical analysis container and `StateSequence` the representation view, recorded as ADR-0002 — every loader returns a pool, and the `coerce` seam lets both containers flow through the statistics and filters layers without churn. Remaining pre-1.0 work is narrower: routing discrepancy analysis through `DistanceMatrix` instead of raw arrays, a preparation seam inside the distance engine, and settling where cluster-quality indices live.

IF YOU READ ONE THING

yasqat brings TraMineR-class sequence analysis to fast, DataFrame-native Python. It is usable today at v0.5.0 — a smaller, sharper engine than the 0.4 line — on the way to a stable 1.0.

| Appendix — Glossary & References

A. Glossary

Sequence — an ordered run of categorical states for one unit (person, account) over time.

Alphabet — the finite set of possible states, with a fixed integer encoding.

Spell — a maximal run of one repeated state within a sequence (e.g. four consecutive months "employed").

STS / SPS / DSS — three standard sequence representations: state-per-timepoint, state-permanence (state + duration), and distinct-successive-states.

Optimal Matching (OM) — an edit-distance between sequences using insertion/deletion (INDEL) and substitution costs; the field-standard dissimilarity.

Substitution cost — the price of swapping one state for another when aligning two sequences; can be constant or derived from transition rates or features.

Medoid — the most central actual sequence in a cluster (PAM clusters around medoids, not synthetic centroids).

Discrepancy analysis — an ANOVA-like test of how much a covariate explains variation in a distance matrix, assessed by permutation.

B. References

- TraMineR (R) — the algorithmic oracle for naming, correctness, and feature scope. traminer.unige.ch
- TanaT (Python) — secondary reference for Python-specific adaptations.
- polars — the columnar DataFrame engine yasqat is built on. pola.rs
- numba & scipy — JIT-compiled metric kernels and hierarchical linkage.
- Source & issue tracker — github.com/rexarski/yasqat
- Documentation — API reference, quick-start, glossary, changelog. rexarski.github.io/yasqat